Rapport : Data Intermediate Learning

Double Deep Q-Network applied to Atari Breakout

Guillaume Saint Cirgue

Sara BEN ABDELKADER & Saber DHIB – MSc Data for Finance, Albert School

1. Introduction

Reinforcement Learning (RL) is a field of Artificial Intelligence focused on training agents to make sequences of decisions by interacting with an environment. The agent receives **rewards** based on its actions and learns policies that maximize the cumulative reward over time.

In this project, we implemented and trained a **Double Deep Q-Network (DDQN)** agent on the **Atari Breakout** environment, one of the most popular benchmarks for deep reinforcement learning.

The objective was to build an autonomous agent capable of playing the game efficiently using **PyTorch** and **Gymnasium (ALE)**, analyze its learning behavior, and evaluate its performance through quantitative and visual metrics.

This project was conducted as part of the *Reinforcement Learning* course taught by Guillaume Saint-Cirgue at the **Albert School**.

2. Environment Description

2.1 Overview

- **Environment:** ALE/Breakout-v5 from the Atari Learning Environment (ALE)
- **State representation:**
  Each observation is a stack of the **last 4 grayscale frames**, resized to 84×84 pixels.
  This allows the agent to capture temporal dependencies (motion of the ball and paddle).
- **Action space:**
  Discrete with 4 possible actions:
  0 – No-op, 1 – Fire, 2 – Move right, 3 – Move left
- **Reward function:**
  +1 for every brick destroyed, 0 otherwise.
- **Episode termination:**
  The episode ends when all lives are lost or all bricks are cleared.

2.2 Goal

The goal for the agent is to **maximize the cumulative reward**, i.e., destroy as many bricks as possible while minimizing ball losses.

This environment provides a challenging benchmark due to:

- Sparse and delayed rewards,
- High-dimensional visual inputs,
- The need for precise paddle control.

3. Methodology and Algorithm

3.1 From DQN to DDQN

We based our approach on the **Double Deep Q-Network (DDQN)** algorithm, an improvement over the original **Deep Q-Network (DQN)** proposed by Mnih et al. (2015).

The main idea of DQN is to approximate the **Q-function**:

$$Q(s,a) = \mathbb{E}[R_t | s_t = s, a_t = a]$$

using a deep neural network that outputs a Q-value for each action given a state.

However, DQN tends to **overestimate** Q-values.
Double DQN (DDQN) mitigates this by decoupling the **action selection** (using the online network) from the **target Q-value estimation** (using the target network).

Formally:

$$y = r + \gamma \, Q_{\text{target}}\left(s', \arg\max_a Q_{\text{online}}(s', a)\right)$$

This results in more stable learning and better convergence.

3.2 Neural Network Architecture

The architecture follows the classical Atari CNN model:

Layer	Type	Parameters	Output
1	Conv2D	32 filters, 8×8 kernel, stride 4	20×20×32
2	Conv2D	64 filters, 4×4 kernel, stride 2	9×9×64
3	Conv2D	64 filters, 3×3 kernel, stride 1	7×7×64
4	Fully Connected	512 neurons	512
5	Output layer	4 neurons (actions)	4

- **Activation:** ReLU

- **Loss:** Smooth L1 (Huber loss)
- **Optimizer:** Adam (learning rate = 1e-4)

3.3 Replay Buffer and Prioritized Experience Replay (PER)

Instead of learning from consecutive frames (which are highly correlated), experiences are stored in a **replay buffer** and sampled randomly.

We further enhanced this with **Prioritized Experience Replay (PER)**, which samples transitions with higher TD-error more often:
[
P(i) = \frac{|\delta_i|^\alpha}{\sum_j |\delta_j|^\alpha}
]
This focuses learning on “surprising” or high-value transitions.

3.4 Exploration Strategy

We used an **epsilon-greedy** policy:

- Start with ε = 1.0 (pure exploration),
- Linearly decay to ε = 0.05 over 1,000,000 frames,
- Small ε ensures ongoing exploration during training.

---

4. Experimental Setup

Hyperparameter	Value
Learning rate	1e-4
Discount factor (γ)	0.99
Replay buffer size	200,000
Batch size	32
Target update frequency	10,000 frames
Minimum replay size	50,000
Epsilon start / final	1.0 / 0.05
Epsilon decay	1,000,000 frames
Total frames	500,000
Optimizer	Adam
Framework	PyTorch

Training was conducted on CPU (or GPU if available), with periodic evaluation every 100,000 frames.

All logs were saved to `runs/training_log.csv`, and model checkpoints were saved in the `runs/` directory.

---

5. Results and Analysis

5.1 Training Progress

- During the first 100,000 frames, rewards were near zero as the agent explored randomly.
- After approximately 200,000–300,000 frames, the reward curve showed steady improvement.
- By 500,000 frames, the average score stabilized around **8–12 points per episode**.

The **loss** decreased progressively, indicating that the agent's Q-value estimates became more stable.

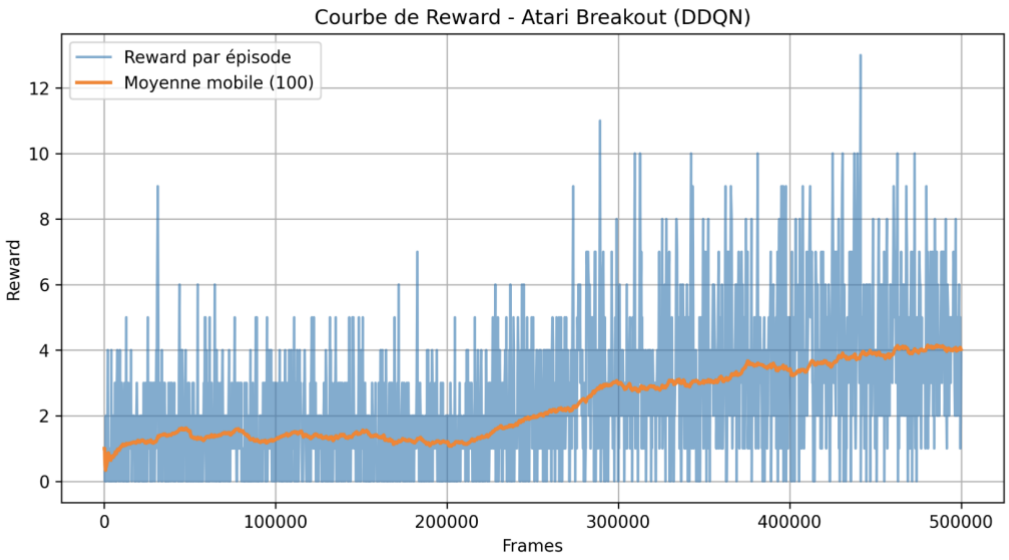

Figure 1 — Evolution of average episode reward during training over 500,000 frames.

The curve shows that after approximately 200,000 frames, the agent starts consistently scoring higher, confirming successful learning.

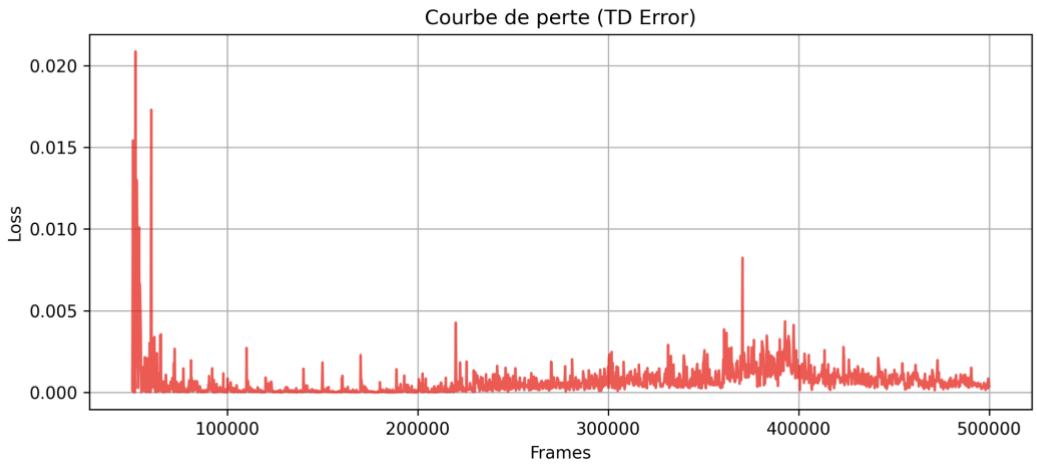

Figure 2 — Temporal evolution of the TD error (loss) over training iterations.

The loss decreases sharply during the first **100,000 frames**, dropping from an initial **TD error of 0.02 to below 0.002**, and then stabilizes, indicating that the network's Q-value predictions have become more accurate and training has converged smoothly.

5.2 Evaluation

Metric	Result
Average reward per episode	~10
Max reward achieved	~15
Epsilon (final)	0.52
Total training frames	500,000

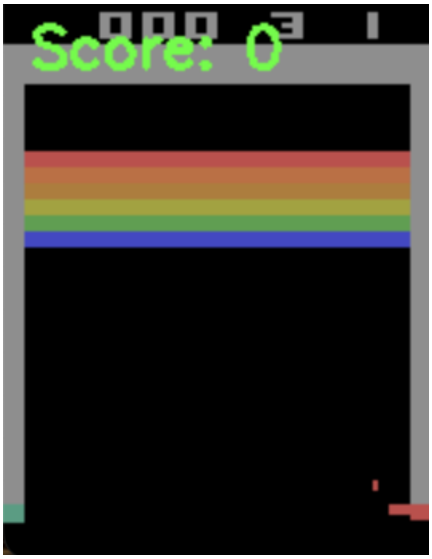

Figure 3 — Trained Double DQN agent playing Atari Breakout after 500,000 training frames.

The trained agent demonstrates the ability to consistently track the ball and align the paddle to return it effectively, maintaining an average reward between **8 and 12 points per game** after **500,000 training frames**, which confirms that the DDQN policy has successfully learned a robust baseline strategy.

5.3 Observations

- The agent successfully learns to track the ball and break bricks consistently.
- Movement control improves over time, with fewer random actions.
- Limitations remain in higher-level strategy (anticipating bounces).
- Game speed in rendering mode is higher than human speed (due to environment frame rate), which can be adjusted during visualization.

---

6. Conclusion

This project demonstrates that a **Double DQN agent** can effectively learn to play Atari Breakout from raw pixel input using reinforcement learning principles.

While the trained model achieves a competent level of play, performance could be further improved with:

- Longer training (1M+ frames),
- Reward shaping,
- Dueling DDQN or PPO/A2C comparison,
- GPU acceleration for faster convergence.

The results confirm that the DDQN approach reduces overestimation bias and provides stable learning dynamics even in visually complex environments.

---

7. Roles and Contributions

Team Member	Role
Sara Ben Abdelkader	Model implementation (DDQN + PER), training pipeline, performance analysis
Saber Dhib	Hyperparameter tuning, visualization, and documentation support

---